



Universidad de
San Andrés

Big Data

Trabajo Práctico 4

Otoño 2026

Integrantes:

Constanza María Efkhanian
Julián Fabrizio Notario
María Florencia Pascale

Fecha de entrega: 29 de mayo de 2026

A. Modelo de Regresión Logística con Regularización: Ridge y LASSO

1. Visualización

Para este ejercicio, se estimaron modelos de regresión logística con penalidad L1 (LASSO) y L2 (Ridge). Las variables fueron estandarizadas con *StandardScaler* antes de la estimación, requisito para que la penalización sea comparable entre predictores, pero solo para las variables numéricas.

A continuación, se muestran las trayectorias de los coeficientes de penalidad de LASSO y Ridge.

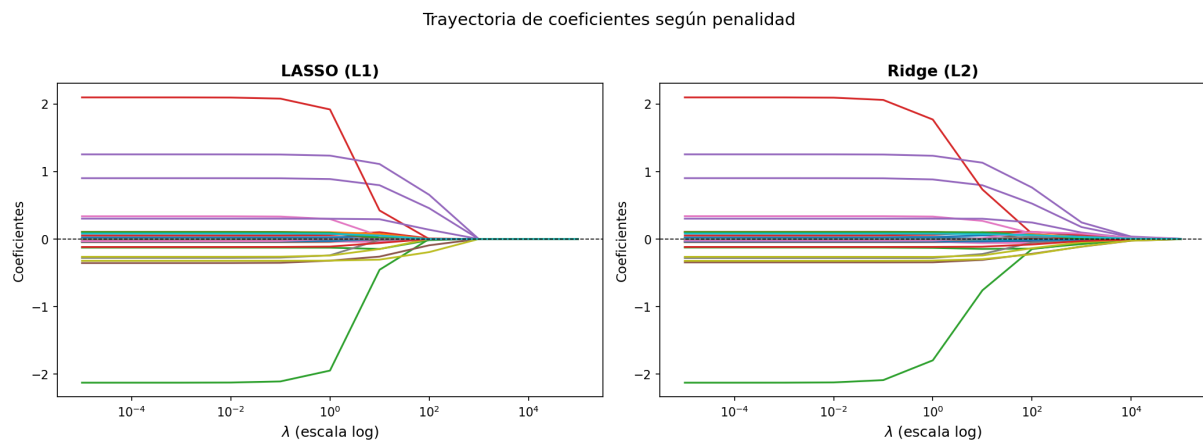


Figure 1: Coeficientes de la penalidad de LASSO y Ridge.

Como se observa, para λ pequeño (izquierda), los coeficientes toman sus valores sin restricción, mientras que al aumentar λ , la penalización los reduce hacia cero. La diferencia cualitativa entre penalidades es clara: LASSO lleva los coeficientes exactamente a cero de forma abrupta (selección de variables), mientras que Ridge los reduce de manera continua y suave sin anularlos. Las variables con mayor señal (*CH06*, *edad2*, *CAT_OCUP*) son las últimas en ser encogidas en ambos casos.

2. Penalidad óptima por Cross-validation y visualización

A continuación, se encuentra en la Figura 2 boxplots de la distribución del MSE para cada λ , y en la Figura 3 el inciso opcional que muestra la proporción de coeficientes equivalentes a 0 según λ .

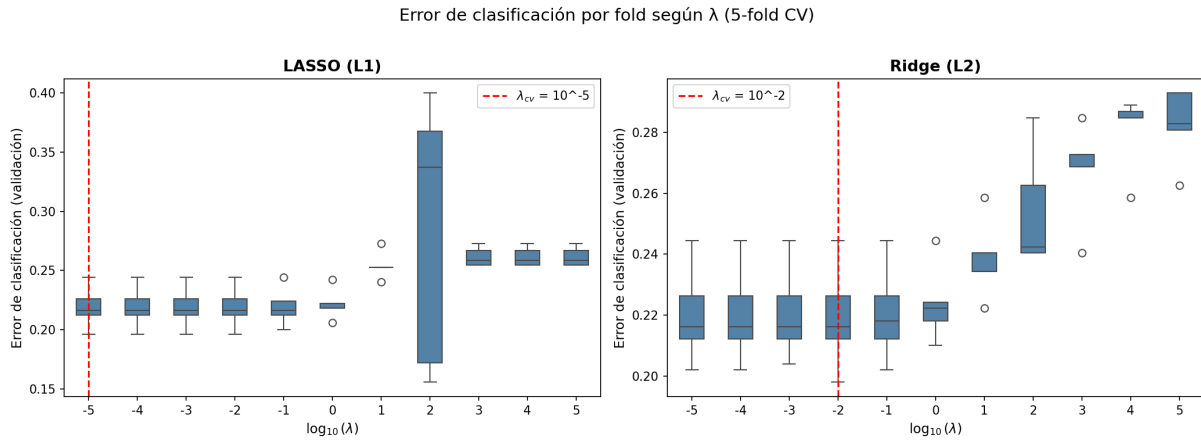


Figure 2: Boxplots de la distribución del error de predicción para cada λ .

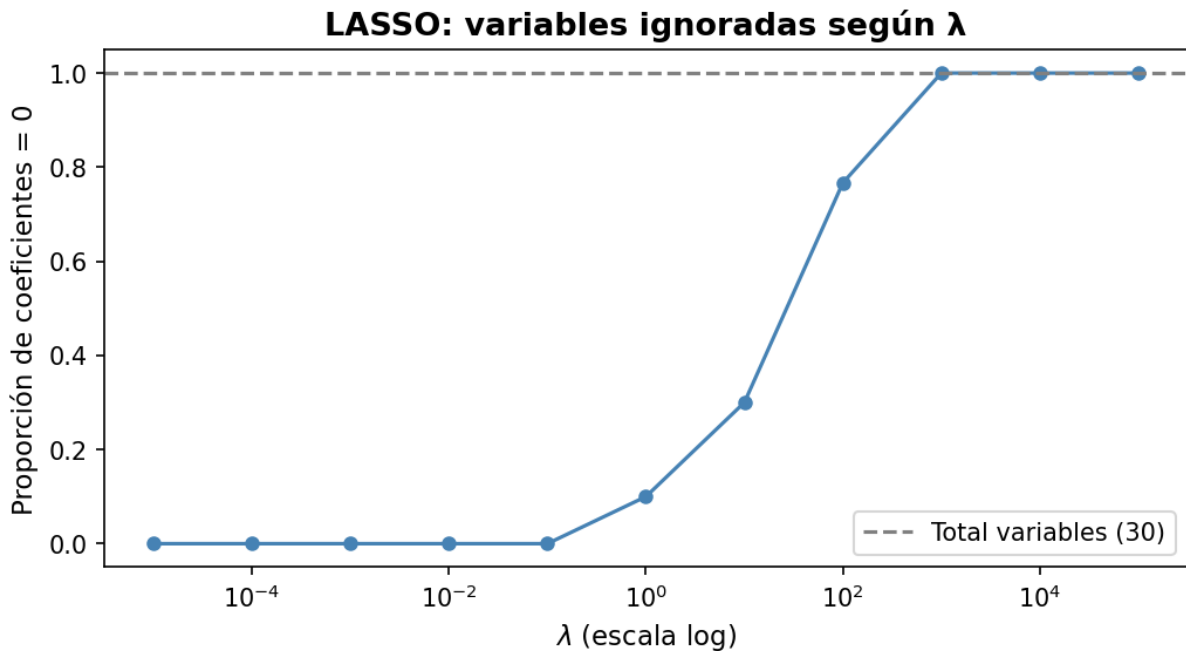


Figure 3: Proporción de coeficientes iguales a cero en LASSO según λ .

Penalizaciones óptimas seleccionadas:

- **LASSO:** $\lambda_{cv} = 10^{-5}$ (mínimo)
- **Ridge:** $\lambda_{cv} = 10^{-2}$

Para LASSO, el *cross-validation* seleccionó la penalización mínima disponible en la grilla. Esto indica que regularizaciones más fuertes aumentan el error de validación, y que el modelo logístico sin regularización es adecuado para estos datos. Es decir, no existe sobreajuste significativo.

Para Ridge, $\lambda_{cv} = 10^{-2}$ representa una reducción leve.

El gráfico de proporciones de ceros confirma que con $\lambda_{cv} = 10^{-5}$ LASSO no anula ninguna variable, mientras que a partir de $\lambda = 10^0$ comienza la selección, y con $\lambda \geq 10^2$ prácticamente todos los coeficientes son cero.

3. Estimación con λ^{cv} y comparación de coeficientes

Se estimaron tres modelos sobre X_{train} : regresión logística sin penalidad ($C = 10^{10}$), con penalidad L1 ($\lambda_{cv} = 10^{-5}$) y con penalidad L2 ($\lambda_{cv} = 10^{-2}$).

Table 1: Comparación de coeficientes estimados

Variable	Logit	LASSO ($\lambda_{cv} = 10^{-5}$)	Ridge ($\lambda_{cv} = 10^{-2}$)
CH03	0.0241	0.0243	0.0249
CH04	-0.0472	-0.0475	-0.0473
CH06	-2.1285	-2.1257	-2.0907
CH07	0.0520	0.0519	0.0533
CH08	0.9020	0.9009	0.8998
CH09	-0.0449	-0.0448	-0.0448
CH10	-0.0358	-0.0361	-0.0365
CH11	-0.0400	-0.0399	-0.0399
CH12	0.0091	0.0100	0.0091
CH13	0.0061	0.0052	0.0059
CH15	0.0175	0.0172	0.0172
CH16	-0.0196	-0.0195	-0.0194
NIVEL_ED	-0.1259	-0.1272	-0.1266
edad2	2.0986	2.0956	2.0611
CAT_OCUP	1.2548	1.2536	1.2522
INTENSI	0.1077	0.1071	0.1077
PP03C	0.3385	0.3365	0.3383
PP03D	-0.2817	-0.2787	-0.2817
PP3E_TOT	-0.2637	-0.2630	-0.2640
PP3F_TOT	0.0026	0.0019	0.0022
PP03G	-0.0370	-0.0369	-0.0372
PP03H	0.1040	0.1039	0.1038
PP03I	0.1013	0.1018	0.1015
PP03J	-0.1175	-0.1181	-0.1176
PP04A	0.3021	0.3021	0.3021
PP04B1	-0.3457	-0.3555	-0.3459
PP04B2	-0.0239	-0.0317	-0.0240
PP04C	-0.0017	-0.0014	-0.0017
PP04G	-0.3238	-0.3222	-0.3231
AGLOMERADO	0.0802	0.0800	0.0800
Error test	0.2024	0.2024	0.2015

Los coeficientes de los tres modelos son bastante similares entre sí. Esto es consecuencia directa de la penalidad mínima seleccionada para LASSO. Las diferencias más visibles se observan en Ridge para las variables de mayor magnitud: *CH06* pasa de -2.129 a -2.091 y *edad2* de 2.099 a 2.061 . Esto refleja la reducción suave típica de L2.

En general, los coeficientes regularizados son menores en magnitud que el logit sin penalidad, aunque marginalmente, dado el λ_{cv} pequeño. Ridge muestra una reducción consistente en todas las variables; LASSO prácticamente no difiere del logit sin penalidad.

Por otro lado, Ridge no elimina variables. Los 30 coeficientes Ridge son distintos de cero, en línea con su propiedad teórica: Ridge reduce los coeficientes pero nunca los anula totalmente. LASSO tampoco elimina variables en este caso, dado que el λ_{cv} seleccionado es casi nulo.

B. Herramientas de IA y reflexión final

1.

Para el presente trabajo, se utilizó únicamente ayuda de la inteligencia artificial Claude. A continuación, se encuentran los prompts utilizados.

Dame las funciones que necesito saber para poder resolver el punto A1 usando las de las tutoriales 8 y 9. Decime como se usan y que hacen.

Las funciones relevantes de las tutoriales para A1 son:

`LogisticRegression(penalty, C, solver, fit_intercept, max_iter)` — de `sklearn.linear_model`

La tutorial menciona que para clasificación se usa esta función con ambas penalidades. Los parámetros clave:

- `penalty='l1'` o `penalty='l2'` — tipo de regularización
- `C=1/lambda` — **inversa** de la fuerza de penalización (a mayor C, menor penalización)
- `solver='liblinear'` para L1, `solver='lbfgs'` para L2
- `fit_intercept=False` — según consigna
- `max_iter=1000` — para asegurar convergencia

Se usa así:

```
python
m = LogisticRegression(penalty='l1', C=1/lam, solver='liblinear',
                       fit_intercept=False, max_iter=1000)
m.fit(X_train_sc, y_train)
coefs = m.coef_[0] # array de shape (n_features,)
```

`StandardScaler()` — de `sklearn.preprocessing`

La tutorial lo aplica antes de regularizar. Estandariza cada variable a media 0 y desvío 1, necesario para que la penalización afecte a todos los coeficientes por igual independientemente de la escala de cada variable.

Se usa así:

```
python
sc = StandardScaler()
X_train_sc = sc.fit_transform(X_train) # fit solo en train
X_test_sc = sc.transform(X_test)      # transform en test con la misma escala
```

Loop sobre grilla + `ax.plot(alphas, coefs)` + `ax.set_xscale("log")` — patrón de la tutorial 9

La tutorial itera sobre la grilla de alphas, guarda los coeficientes en una lista, y los grafica todos juntos. El eje x va en escala logarítmica.

```
python
alphas = 10**np.linspace(-5, 5, 11)
coefs = []
for a in alphas:
    m = LogisticRegression(penalty='l2', C=1/a, fit_intercept=False)
    m.fit(X_train_sc, y_train)
    coefs.append(m.coef_[0])
```

Dame las funciones que necesito saber para poder resolver el punto A2 usando las de las tutoriales 8 y 9. Decime como se usan y que hacen.

Las funciones relevantes de las tutoriales para A.2 son:

`KFold(n_splits, shuffle, random_state)` — de `sklearn.model_selection`

La tutorial 7 la usa para implementar el CV manualmente. Divide el conjunto de entrenamiento en K partes, generando en cada iteración índices de train y validación.

Se usa así:

```
python
kf = KFold(n_splits=5, shuffle=True, random_state=100)

for i, (train_idx, val_idx) in enumerate(kf.split(X_train_sc)):
    X_fold_tr = X_train_sc[train_idx]
    X_fold_val = X_train_sc[val_idx]
    y_fold_tr = y_train.iloc[train_idx]
    y_fold_val = y_train.iloc[val_idx]
```

Cada iteración da un fold distinto como validación y el resto como entrenamiento — el esquema OXXXX/XOXXX/ etc. de la tutorial. ↓

`pd.DataFrame` + `pd.concat` — patrón de la tutorial 7

La tutorial guarda los resultados de cada fold en un DataFrame acumulativo con columnas ["grado", "particion", "mse"]. Para A.2 el esquema es el mismo pero con ["lambda", "particion", "error"]:

```
python
resultados = pd.DataFrame(columns=["lambda", "particion", "error"])

for lam in lambdas:
    for i, (train_idx, val_idx) in enumerate(kf.split(X_train_sc)):
        # ... ajustar modelo, calcular error ...
        df_i = pd.DataFrame({
            "lambda": [lam],
            "particion": [i],
            "error": [error]
        })
        resultados = pd.concat([resultados, df_i], ignore_index=True)
```

`sns.boxplot(data, x, y)` — patrón de la tutorial 7

La tutorial usa exactamente esta función para visualizar la distribución del error por valor del parámetro, con el parámetro en el eje x y el error en el eje y:

```
python
sns.boxplot(data=resultados, x="log_lambda", y="error", ax=ax)
```

Selección del óptimo por mínimo error promedio — patrón de la tutorial 7

La tutorial elige el mejor modelo calculando el error promedio por valor del parámetro y buscando el mínimo:

```
python
avg = resultados.groupby("log_lambda")["error"].mean()
lam_opt_log = avg.idxmin()
```

Dame las funciones que necesito saber para poder resolver el punto A3 usando las de las tutoriales 8 y 9. Decime como se usan y que hacen.

Las funciones relevantes de las tutoriales para A.3 son:

LogisticRegression(C=1e10) — logit sin penalidad

La tutorial usa **Ridge(alpha=0)** para el modelo sin regularización. Para clasificación el equivalente es poner C muy grande, lo que hace la penalización prácticamente nula:

```
python
m_logit = LogisticRegression(penalty='l2', C=1e10, solver='lbfgs',
                             fit_intercept=False, max_iter=1000)
m_logit.fit(X_train_sc, y_train)
```

.coef_[0] — extraer coeficientes

La tutorial extrae los coeficientes con **.coef_** y los asocia a los nombres de columnas con **pd.Series**:

```
python
coefs_logit = m_logit.coef_[0]
```

```
python
coefs_logit = m_logit.coef_[0]
coefs_lasso = m_lasso.coef_[0]
coefs_ridge = m_ridge.coef_[0]
```

pd.DataFrame para construir la tabla comparativa — patrón de la tutorial 9

La tutorial arma la tabla de coeficientes exactamente así, con una columna por modelo:

```
python
df_coef = pd.DataFrame({
    'Variable': X_train.columns,
    'Logit_sin_pen': coefs_logit,
    'LASSO_L1': coefs_lasso,
    'Ridge_L2': coefs_ridge,
})
```

pd.concat — agregar filas de métricas

La tutorial agrega el intercepto y el MSE como filas extra con **pd.concat**. Para A.3 se agrega el error de test:

La tutorial agrega el intercepto y el MSE como filas extra con **pd.concat**. Para A.3 se agrega el error de test:

```
python
error_df = pd.DataFrame({
    'Variable': ['Error_test'],
    'Logit_sin_pen': [1 - m_logit.score(X_test_sc, y_test)],
    'LASSO_L1': [1 - m_lasso.score(X_test_sc, y_test)],
    'Ridge_L2': [1 - m_ridge.score(X_test_sc, y_test)],
})

df_comp = pd.concat([df_coef, error_df], ignore_index=True).round(4)
```

np.isclose — identificar coeficientes en cero de LASSO

La tutorial usa exactamente esta función para detectar qué variables anuló LASSO:

```
python
mascara_cero = np.isclose(coefs_lasso, 0, atol=1e-8)
variables_cero = X_train.columns[mascara_cero].tolist()
print("Variables con coef LASSO = 0:", variables_cero)
```

2.

La inteligencia artificial nos fue útil en la corrección de código, en la limpieza y en el uso de diversos comandos. Para la interpretación y redacción del informe, preferimos confiar en nuestro propio criterio.

3.

Dedicamos, aproximadamente, 5 horas al trabajo.